# Security Protocols

## Module 4, Part 2/2: SSL/TLS and Email Security

## CSC 3114: Cryptology and Coding Theory

Denish Azamuke, PhD

Office: CoCIS, Block B, Level 3

October 19, 2026

# Module 4

**Part 1: Previous lecture**

- Security protocol foundations.
- Authentication and freshness.
- IPsec: AH and ESP.
- Transport and tunnel modes.
- Security associations and IKEv2.

**Part 2: This lecture**

- SSL background and TLS.
- TLS authentication and secure sessions.
- Email signatures and encryption.
- S/MIME and OpenPGP.
- Protection at different layers.

# Learning Outcomes

By the end of this lecture, you should be able to:

1. Distinguish obsolete SSL from TLS.
2. Trace a simplified TLS 1.3 handshake.
3. Explain certificate validation and proof of private-key possession.
4. Explain record protection, forward secrecy, and session resumption.
5. Describe how signatures and hybrid encryption protect email.
6. Select IPsec, TLS or message security for a stated requirement.

# SSL Background and TLS

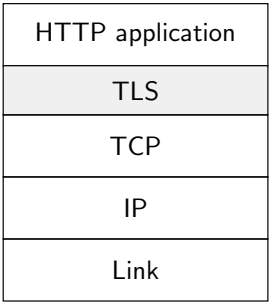
**SSL**: Secure Sockets Layer. **TLS**: Transport Layer Security.

| Protocol | Status |
|---|---|
| SSL 2.0 and 3.0 | Obsolete. Disable them. |
| TLS 1.0 and 1.1 | Deprecated. Disable them. |
| TLS 1.2 | Requires careful configuration where interoperability needs it. |
| TLS 1.3 | Our main example of an authenticated secure channel. |

### Terminology

The phrase "SSL certificate" often refers to an X.509 certificate used with TLS. The certificate and the TLS protocol version are different things.

# TLS in the TCP/IP Stack




HTTPS combines HTTP with a secure transport.

TLS authenticates the peer and protects application bytes on this connection.

TCP carries the protected bytes.
IP routes the packets.

# TLS Security Goals and Boundaries

**A correctly established channel provides**

- Peer authentication under the chosen trust model.
- Confidentiality of protected application data.
- Detection of changes to protected data.

**The application still handles**

- User login and authorisation.
- Data before encryption and after decryption.
- Storage, malware and compromised endpoints.

Protection boundary

TLS protection ends at the TLS endpoints. IP addresses, traffic timing and approximate sizes may remain observable.

# TLS Components

| Component | Purpose |
|---|---|
| Handshake protocol | Negotiate parameters, authenticate peers and establish shared keying material. |
| Record protocol | Divide data into records and protect them using traffic keys. |
| Alerts | Signal errors and orderly closure. |

## Connection to Part 1

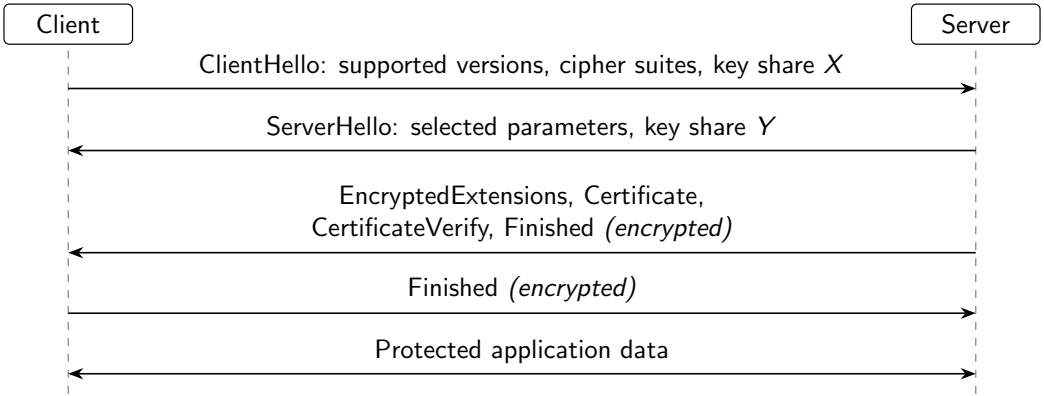
The handshake combines key establishment with authentication. The record protocol then protects application traffic.

# Notation for the TLS Examples

| | |
|---|---|
| $C, S$ | Client and server. |
| $x, y$ | Fresh ephemeral private values. |
| $X, Y$ | Corresponding public key shares. |
| $Z$ | Shared Diffie–Hellman secret. |
| $T$ | Handshake transcript up to the relevant point. |
| $H(T)$ | Cryptographic hash of that transcript. |
| HKDF | HMAC-based key derivation function. |
| AEAD | Authenticated encryption with associated data. |

Our main handshake uses certificates and ephemeral Diffie–Hellman. PSK-based variants appear later.

# TLS 1.3: Simplified Full Handshake




Server authentication only. Client certificates, retries, and optional messages are omitted.

# Ephemeral Diffie–Hellman and Key Derivation

**Client** **Server**

$X = g^x \bmod p$, $Z = Y^x \bmod p$. $Y = g^y \bmod p$, $Z = X^y \bmod p$.

- Both obtain $Z = g^{xy} \bmod p$ without transmitting $x$, $y$ or $Z$.
- HKDF combines the shared secret with transcript context to derive separate traffic keys.
- Certificate authentication binds the exchange to the intended server.

## Elliptic-curve version

ECDHE uses points: $X = xG$, $Y = yG$ and $Z = xY = yX$. The certificate key signs the exchange rather than encrypting the session key.

Equations demonstrate the shared-secret step.

# Example: A Shared Secret

Let $p = 23$, $g = 5$, $x = 6$ and $y = 15$.

**Client sends**

$$X = 5^6 \bmod 23 = 8.$$

**Server sends**

$$Y = 5^{15} \bmod 23 = 19.$$

**Client computes**

$$Z = 19^6 \bmod 23 = 2.$$

**Server computes**

$$Z = 8^{15} \bmod 23 = 2.$$

What this calculation establishes

Both obtain the same value. Authentication is still required to stop an active attacker from substituting key shares.

These tiny values are insecure. TLS derives traffic keys using approved groups and HKDF.

# Certificates and the Chain of Trust




- A certificate binds identifiers to a public key under a CA's signature.
- The client validates a path to a trust anchor it already accepts.
- Merely receiving a root certificate does not make that root trusted.

### Certificate versus live proof

The certificate provides a key binding. CertificateVerify shows that the peer controls the matching private key in this handshake.

CA = certification authority.

# Certificate Validation: A Portal Example

Expected service: `portal.campus.example`.

- Validate the certification path, time validity and permitted key use.
- Match the expected service name to the certificate's subjectAltName.
- Apply algorithm restrictions and applicable certificate-status policy.

| **Certificate evidence** | **Decision** |
|---|---|
| Trusted path, matching name, valid time and other checks pass | Accept this certificate. |
| Trusted path, but only `mail.campus.example` appears | Reject: name mismatch. |
| Matching name, but the certificate has expired | Reject: validity failure. |
| Self-signed certificate with no configured trust binding | Reject: no trusted path. |

Certificate acceptance alone does not complete the TLS handshake.

# CertificateVerify and Finished

## CertificateVerify

The server signs a role context and the current transcript hash:

$$\sigma = \mathrm{Sign}_{sk_S}\big(\text{context} \parallel H(T)\big).$$

The client verifies using the validated certificate key. This proves possession of the matching private key.

## Finished

Each peer authenticates its current transcript using a derived key:

$$v = \mathrm{HMAC}_{K_{\mathrm{finished}}}\big(H(T)\big).$$

The expected result confirms key possession and the integrity of the negotiated exchange.

$T$ grows with the handshake, so each proof covers a different transcript prefix.
**Note:** The signature formula omits the protocol's exact encoding.

# Server Authentication and Mutual TLS

**Server authentication**

- The client validates the server's certificate and handshake proof.
- The server may authenticate the user later through an application login.

**Mutual TLS (mTLS)**

- The server also requests a client certificate and signature proof.
- Both sides validate the expected identity or credential binding.

University systems

A student portal may use server-authenticated TLS plus user login. An internal service may require mTLS certificates for its calling services.

Authorisation still determines what an authenticated user or service may do.

# TLS 1.3 Cipher Suites

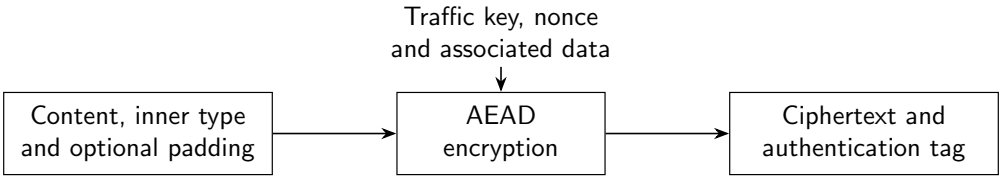
`TLS_AES_128_GCM_SHA256`

| Element | Meaning |
|---|---|
| AES_128_GCM | AEAD record protection using a 128-bit AES key. |
| SHA256 | Hash for HKDF and the handshake transcript. |
| Key-exchange group | Negotiated separately, for example X25519. |
| Signature scheme | Negotiated separately, for example RSA-PSS or ECDSA. |

**Separation of choices**

A TLS 1.3 cipher-suite name identifies the record algorithm and hash. It does not identify the handshake signature scheme or key-exchange group.

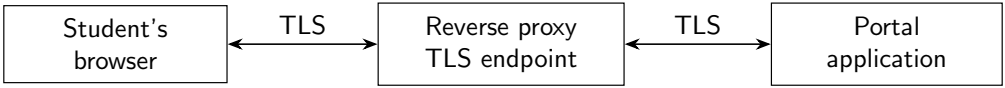
# TLS 1.3 Record Protection




- The outer record header supplies associated data and remains visible.
- The receiver verifies the tag before accepting the recovered content.
- A failed authentication check terminates the connection.

Associated data receives integrity protection without encryption.

# HTTPS Example: A University Portal




- The browser authenticates the expected portal service.
- The proxy decrypts requests and can read their contents.
- A separate protected backend connection has its own keys and peer checks.
- The application still checks login, permissions and duplicate transactions.

## Design observation

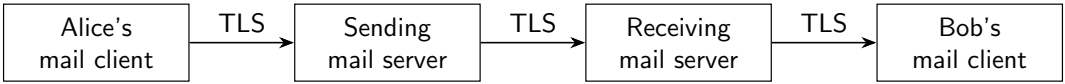
Two TLS connections create two protection boundaries. The proxy is a trusted processing point.

# Email Security: Threats and Goals

| **Threat** | **Relevant protection** |
|---|---|
| Eavesdropping on a connection | TLS for that connection. |
| Reading content at a mail server | Message encryption with keys outside that server. |
| Changing protected message content | A verifiable digital signature or authenticated encryption. |
| Impersonating a sender | Trusted sender-key binding and appropriate domain checks. |
| Reusing a valid old message | Application context and duplicate detection. |

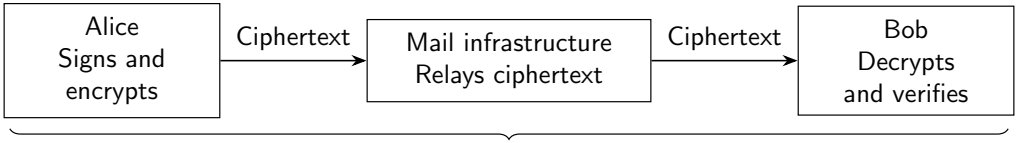
A signed or encrypted message can still contain misleading information or a harmful attachment.

# Email TLS: Protection on Each Connection




Each server terminates a connection and may process or store plaintext.

# End-to-End Email Content Protection




- S/MIME and OpenPGP can protect the message object across relays.
- Ciphertext can remain protected while stored at a mail server.
- Routing metadata and outer headers may remain visible.

If an organisational gateway holds the decryption keys, that gateway becomes a content-protection endpoint.

# Digital Signatures for Email

Let $M$ be the message content covered by the signature.

$$\sigma = \mathrm{Sign}_{sk_A}(M), \qquad \mathrm{Verify}_{pk_A}(M, \sigma) \in \{\mathrm{valid}, \mathrm{invalid}\}.$$

- Alice signs with her private signing key.
- Bob verifies using Alice's public key after checking its identity binding.
- A valid signature detects changes to the covered bytes and supports attribution to the signing key.

## Meaning of a valid signature

It does not hide $M$, establish freshness or prove that every outer email header is authentic. Key control and identity checks determine whom Bob should trust.

Sign denotes the complete signature algorithm, including its hashing rules.

# Hybrid Encryption for Email

1. Generate fresh random keying material $K$ for the message.
2. Encrypt the content using symmetric authenticated encryption.
3. Protect $K$ separately for each intended recipient using that recipient's public key.

$$C = \mathrm{AEAD.Enc}_K(M), \qquad W_B = \mathrm{Protect}_{pk_B}(K).$$

**Bob's recovery steps**

Use $sk_B$ to recover $K$ from $W_B$. Then verify and decrypt $C$.

Conceptual notation: formats specify key encapsulation or wrapping, derived content keys, nonces, and associated data. The complete message uses these rules rather than raw public-key encryption.

# S/MIME

**Secure/Multipurpose Internet Mail Extensions** adds cryptographic protection to MIME content.

- Uses Cryptographic Message Syntax (CMS) for signed or encrypted objects.
- Commonly uses X.509 certificates for sender and recipient public keys.
- Supports signatures and encryption, separately or in combination.
- Requires certificate validation and the expected email identity binding.

**University deployment**

An institution can issue or obtain certificates for staff and manage renewal, revocation and private-key protection.

Sender signatures and recipient encryption may use different keys and certificates.

# OpenPGP

**OpenPGP** defines formats for encrypted messages, signatures and transferable keys.

- Uses symmetric content encryption with protected session-key entries.
- A key may carry user IDs and certifications from other keys.
- Fingerprints identify a specific public key.
- The application or organisation decides which identity bindings to trust.

### Public-key substitution

A key carrying "Alice" or her email address does not establish that Alice owns it. Verify the fingerprint through a trusted channel or use an established trust policy.

Revocation, expiry, backups, and protection of private keys remain operational responsibilities.

# Design Examples: Choosing the Protection

| Requirement | Suitable mechanism |
|---|---|
| Protect traffic between two campus gateways. | IPsec ESP tunnel with IKEv2. |
| Protect a browser connection to the student portal. | TLS with service-identity validation. |
| Let recipients verify the origin of a staff notice. | Message signature with a trusted sender key. |
| Keep a sensitive attachment confidential across mail relays and in server storage. | Message encryption to verified recipient keys, with a signature when sender authentication is needed. |

User permissions, private-key management and endpoint security support every design.

# Questions?